

Esercizio A1

Andando per tentativi, proviamo con $\Theta(n^2)$. È facile vedere che la ricorrenza è $\Omega(n^2)$, per via della sua componente non ricorsiva. Proviamo quindi a dimostrare che $T(n) = O(n^2)$.

- Caso base: $T(n) = 1 \leq cn^2$, per tutti i valori di n compresi fra 1 e 6, ovvero:

$$c \geq 1, c \geq \frac{1}{4}, c \geq \frac{1}{9}, c \geq \frac{1}{16}, c \geq \frac{1}{25}, c \geq \frac{1}{36}$$

Tutte queste disequazioni sono soddisfatte da $c \geq 1$.

- Ipotesi induttiva: $T(k) \leq ck^2$, per $k < n$
- Passo induttivo:

$$\begin{aligned} T(n) &= 3T(\lfloor n/3 \rfloor) + 4T(\lfloor n/4 \rfloor) + 12T(\lfloor n/6 \rfloor) + n^2 \\ &\leq 3c\lfloor n/3 \rfloor^2 + 4c\lfloor n/4 \rfloor^2 + 12\lfloor n/6 \rfloor^2 + n^2 \\ &\leq 3cn^2/9 + 4cn^2/16 + 12cn^2/36 + n^2 \\ &\leq 11/12cn^2 + n^2 \leq cn^2 \end{aligned}$$

L'ultima disequazione è rispettata per $c \geq 12$.

Abbiamo quindi dimostrato che $T(n) = \Theta(n^2)$, con $m = 1$ e $c \geq 12$.

Esercizio A2

L'esercizio è simile all'esercizio del compito scorso, che avevamo risolto interpretando la matrice come un grafo e facendo una visita BFS visto che erano coinvolte le distanze. In questo caso, risolviamo il problema tramite una DFS - ne risulterà una soluzione più compatta.

Utilizziamo la matrice M di input come vettore *visited*; in altre parole, una cella è visitabile se il suo valore è maggiore di zero. Una volta scoperta, il valore viene posto a zero. Se la matrice deve essere conservata, sarà necessario farne una copia prima (con costo $O(n^2)$).

L'algoritmo scritto qui simula una ricerca delle componenti connesse nel grafo. Si parte da ogni cella non visitata, e si lancia una visita DFS. Per ogni nodo visitato, si aggiunge l'altezza ad un accumulatore di altezze *height*, si aggiunge 1 ad un contatore *count* e si marca il nodo come visitato. Si continua quindi la visita affrontando le quattro caselle vicine. Al termine della visita ricorsiva, si calcola l'altezza media e la si confronta con il massimo.

Visto che ogni cella può essere visita al più una volta, la complessità è $O(n^2)$.

```

int topsland(int[][] M, int n)


---


    float max = 0
    for r = 1 to n do
        for c = 1 to n do
            if M[r][c] > 0 then
                int count = 0
                int height = 0
                dfsRec(M, n, r, c, &count, &height)

                max = max(max, height/count)
            end if
        end for
    end for

    return max


---



```

```
dfsRec(int[][] M, int n, int r, int c, int
count, int height)
```

```
  if  $1 \leq r < n$  and  $1 \leq c < n$  and
```

```
    M[r][c] > 0 then
```

```
      height = height + M[r][c]
```

```
      count = count + 1
```

```
      M[r][c] = 0                                % Visited
```

```
      dfsRec(M, n, r - 1, c, &count, &height)
```

```
      dfsRec(M, n, r + 1, c, &count, &height)
```

```
      dfsRec(M, n, r, c - 1, &count, &height)
```

```
      dfsRec(M, n, r, c + 1, &count, &height)
```

Esercizio A3

Il problema può essere risolto modificando opportunamente la ricerca dicotomica, in cui cerchiamo di individuare la posizione k del minimo assumendo che il vettore sia stato traslato di un fattore k .

L'idea è la seguente: si considera un sottovettore compreso fra gli indici i e j , e si considera l'elemento centrale $m = \lfloor (i + j)/2 \rfloor$.

- Se $A[i] > A[m]$, allora il minimo si troverà fra i ed m , in quanto il sottovettore di sinistra non è ordinato e quindi il minimo si troverà in quel sottovettore; si noti che necessariamente si ha che $A[m] < A[j]$, in quanto se fosse $A[m] > A[j]$ il vettore sarebbe ordinato in senso opposto.
- Se $A[m] > A[j]$, allora il minimo si troverà fra m e j , in quanto il sottovettore di destra non è ordinato e quindi il minimo si troverà in quel sottovettore; si noti che necessariamente si ha che $A[i] < A[m]$, in quanto se fosse $A[i] > A[m]$ il vettore sarebbe ordinato in senso opposto.
- Se entrambe le condizioni non sono vere, allora $A[i] < A[m] < A[j]$, e il vettore è ordinato. In questo caso bisogna restituire 0.

Accorgersi della terza condizione è difficile, anche per l'ambiguità del testo - non viene specificato se k può essere uguale a zero oppure no. Per questo motivo, non ho valutato negativamente soluzioni che non gestiscono questo caso particolare.

Come caso base, quando si è rimasti con due elementi, quindi $j = i + 1$, allora il minimo si trova in j (se l'array fosse ordinato, avremmo già restituito zero) e il numero di shift è pari a $j - 1$ (in vettori che iniziano da 1).

Una dimostrazione più precisa dovrebbe essere basata su induzione sulla dimensione del sottovettore, con caso base 2.

Il codice, molto breve, è il seguente:

```
int shift(int[] A, int i, int j)
{
    if j == i + 1 then
        return j - 1
    else
        int m =  $\lfloor (i + j)/2 \rfloor$ 
        if A[i] > A[m] then
            return shift(A, i, m)
        else if A[m] > A[j] then
            return shift(A, m, j)
        else
            return 0
}
```

La complessità è ovviamente $O(\log n)$, derivando dalla ricerca dicotomica.